ABSTRACT

The automated estimation of age and gender from facial images is crucial for various intelligent applications, including human-computer interaction and marketing, but developing a real-time system from live video presents significant challenges due to the need for a balance between accuracy and computational efficiency. This project proposes a real-time age and gender detection system that addresses these challenges by leveraging pre-trained deep learning models within OpenCV's Deep Neural Network (DNN) module, offering an efficient solution runnable on standard hardware without extensive GPU resources. While numerous deep learning models exist for facial attribute classification, many are computationally intensive and unsuitable for real-time processing on consumer-grade CPUs, and training such models from scratch is a massive undertaking requiring vast datasets and computational power, as exemplified by custom CNNs trained on large datasets like Adience which, despite high accuracy, can be too slow for live video analysis. Therefore, a more efficient strategy involves utilizing pre-trained models optimized for inference, with the main challenge being their integration into a real-time pipeline capable of handling video input, performing face detection, and executing classification efficiently per frame.This project will utilize the OpenCV DNN module to load and run pre-trained Caffe models for age and gender classification through a multi-stage pipeline: first, an optimized deep learning-based face detector from OpenCV will localize faces in each frame from a live webcam feed, offering greater accuracy than traditional Haar cascade classifiers; second, once a face is detected, its bounding box will be used to extract the facial Region of Interest (ROI), which will then be preprocessed (resized and converted to grayscale); third, the preprocessed ROI will be fed into two separate pre-trained deep learning models, one for gender classification ('Male' or 'Female') and the other for age estimation into one of eight predefined ranges (e.g., 0-2, 4-6, 8-12), using compact CNN architectures provided in Caffe framework format (.prototxt and .caffemodel) that can be loaded directly; and finally, predicted age and gender labels will be overlaid. The final deliverable will be a functional Python application for real-time age and gender dection, accompanied by a report detailing its architecture, implementation, and performance.

CONTENTS

S. No	TITLES	PAGE NO
1.	**INTRODUCTION**	**3**
	1.1 SYSTEM SPECIFICATION	**4**
	1.2.1 HARDWARE CONFIGURATION	**4**
	1.2.2 SOFTWARE SPECIFICATION	**4**
2.	**SYSTEM STUDY**	**21**
	2.1 EXISTING SYSTEM	**21**
	2.1.2 DRAWBACKS	**21**
	2.2PROPOSED SYSTEM	**21**
	2.2.1 ADVANTAGES	**21**
3.	**SYSTEM DESIGN AND DEVELOPMENT**	**22**
	3.1 FILE DESIGN	**22**
	3.2.INPUT DESIGN	**22**
	3.3 CODE DESIGN	**23**
	3.4 OUTPUT DESIGN	**23**
	3.3 DATABASE DESIGN	**24**
	3.6 SYSTEM DEVELOPMENT 3.6.1 MODULES 3.6.2 MODULES DESCRIPTION	**25**
4.	**SYSTEM TESTING AND IMPLEMENTATION**	**27**
5.	**FEASIBILITY STUDY**	**31**
6.	**APPENDICES** **A. SYSTEM ARCHITECTURE** **B. TABLE STRUCTURE** **C. SAMPLE SOURCE CODE** **D. SAMPLE** **INPUT SAMPLE** **OUTPUT**	**32**
7.	**CONCLUSION** **REFERENCE**	**48**

1. INTRODUCTION

The human face is a rich source of information, conveying not only identity and emotion but also demographic attributes such as age and gender. The ability for machines to automatically perceive and interpret these attributes in real-time has become a key enabling technology for a wide range of applications. In retail, it can be used for demographic analysis to understand customer bases and tailor marketing efforts. In human-computer interaction, systems can adapt their interface or responses based on the user's age and gender. In social robotics and digital signage, it allows for more personalized and engaging interactions. The task of **Real-time Gender and Age Detection** involves analyzing a live video stream, locating human faces, and for each face, predicting its gender and estimating its age category.However, developing a system that is both accurate and fast enough for real-time applications on standard hardware presents a significant challenge. The visual appearance of a face is highly variable, affected by factors such as pose, illumination, facial expression, and accessories like glasses or makeup. While large-scale, deep Convolutional Neural Networks (CNNs) have achieved remarkable accuracy on this task, they are often computationally very expensive, requiring powerful GPUs for inference, which makes them unsuitable for many practical, low-cost applications. On the other hand, simpler classical methods often lack the necessary accuracy and robustness to handle the variability of real-world facial images.The primary problem this project addresses is the need for a **computationally efficient yet accurate system for real-time age and gender detection from a live video feed**. The core objective is to design and implement a complete pipeline that intelligently leverages **pre-trained, optimized deep learning models** within the highly efficient **OpenCV Deep Neural Network (DNN) module**. Instead of training a massive model from scratch, this project focuses on the practical engineering task of integrating several pre-trained components: a highly accurate deep learning-based face detector to localize faces, and two separate, compact CNNs for the distinct tasks of gender classification and age estimation. This approach allows the system to achieve high performance while running efficiently on a standard CPU, demonstrating a practical and accessible solution for real-world demographic analysis.

1.1 SYSTEM SPECIFICATION

1.2.1 HARDWARE REQUIREMENT

- Processor - Intel core i3
- Speed - 2.4 Ghz
- RAM - 8 GB
- Hard Disk - 1TB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

1.2.2 SOFTWARE CONFIGURATION:-

- Operating system : Windows 10/11
- Programming lang : Python
- IDE : Jupyter Notebook, VSCode
- Tools : Anaconda, Pyenv

SOFTWARE ENVIRONMENT

Python

Python is a high-level, interpreted programming language renowned for its simplicity, readability, and extensive ecosystem of libraries. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with its notable use of significant whitespace. It is a dynamically typed and garbage-collected language that supports multiple programming paradigms, including structured (particularly, procedural), object-oriented, and functional programming. Its clear and concise syntax makes it an excellent choice for beginners, while its power and versatility have made it a dominant force in various fields, including web development, data science, artificial intelligence, and scientific computing.

One of the most significant advantages of Python is its vast and ever-growing standard library, which provides tools for a wide range of tasks. This "batteries-included" philosophy means that developers have access to modules for tasks like working with strings, regular expressions, file I/O, networking, and more, without needing to install external packages. This reduces development time and makes it easier to write robust and maintainable code. The Python Package Index (PyPI) further extends Python's capabilities, hosting hundreds of thousands of third-party packages that cater to almost any conceivable need.

In the context of data science and machine learning, Python has become the de facto standard. The availability of powerful libraries such as NumPy, Pandas, and Scikit-learn has created a comprehensive environment for data manipulation, analysis, and modeling. The language's gentle learning curve allows data scientists and researchers, who may not have a traditional computer science background, to quickly become productive. Furthermore, the interactive nature of tools like Jupyter Notebooks, which are built on Python, allows for an exploratory and iterative approach to data analysis. The ability to mix code, visualizations, and narrative text in a single document makes it an ideal tool for documenting and sharing research. The rise of deep learning frameworks like TensorFlow and PyTorch, both of which have excellent Python APIs, has further solidified Python's position as the leading language for artificial intelligence research and development.

JUPYTER NOTEBOOK

The Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text. Its name is a reference to the three core programming languages it was designed for: Julia, Python, and

R. Jupyter Notebooks have become an indispensable tool for data scientists, researchers, and educators due to their interactive and exploratory nature. They provide a flexible environment for data cleaning and transformation, numerical simulation, statistical modeling, data visualization, and machine learning. The notebook interface is divided into cells, which can contain either code or Markdown text. This cell-based structure allows for an iterative and modular approach to coding, where users can execute small snippets of code independently and see the results immediately. This is in stark contrast to traditional integrated development environments (IDEs) where the entire script is typically run at once.

One of the key features of Jupyter Notebooks is their ability to display output directly below the code cell that produced it. This can include the results of calculations, tables of data, and, most importantly, visualizations. This tight integration of code and output makes it easy to understand the flow of an analysis and to see the impact of code changes in real-time. The support for Markdown in text cells allows for the creation of richly formatted documents that can include headings, lists, links, and even mathematical equations using LaTeX. This makes Jupyter Notebooks an excellent tool for creating reports, tutorials, and presentations.

The architecture of Jupyter Notebooks is based on a two-process model. The user interacts with a web browser, which acts as the front-end interface. The actual code execution is handled by a separate process called a kernel. This separation of concerns allows for a great deal of flexibility. For example, the kernel can be running on a remote server, allowing users to leverage powerful computing resources from a simple laptop. There are kernels available for a wide variety of programming languages, not just Python, Julia, and R. This makes Jupyter a versatile platform for a wide range of computational tasks. The notebook documents themselves are stored as JSON files with a .ipynb extension. This makes them easy to version control and share with others.

ANACONDA

Anaconda is a free and open-source distribution of the Python and R programming languages for scientific computing, that aims to simplify package management and deployment. The distribution includes over 250 popular data science packages and provides access to over 7,500 additional open-source packages. It is one of the most popular platforms for data scientists and is used by millions of users worldwide. The core of the Anaconda distribution is the conda package manager, which is a cross-platform, language-agnostic package and environment manager. Conda makes it easy to install, run, and update packages and their dependencies. It also allows you to create isolated environments that can have different versions of Python and/or packages installed in them. This is particularly useful for managing projects with conflicting dependencies.

One of the main advantages of using Anaconda is that it simplifies the process of setting up a data science environment. Instead of having to manually install Python and all the necessary libraries, you can simply install the Anaconda distribution, which comes with everything you need to get started. This is especially beneficial for beginners who may not be familiar with the intricacies of package management. The Anaconda Navigator, a graphical user interface (GUI) included in the distribution, makes it even easier to manage packages and environments without having to use the command line. From the Navigator, you can launch applications like Jupyter Notebook, Spyder (an IDE for scientific Python), and RStudio.

Beyond package and environment management, Anaconda also provides a number of other tools and services for data scientists. The Anaconda Cloud is a package management service by Anaconda where you can find, access, store and share public and private notebooks, environments, and conda and PyPI packages. It also allows you to stay up-to-date with the latest versions of your favorite packages. For enterprise users, Anaconda offers a commercial version of the distribution that includes additional features such as enhanced security, better package management, and support for large-scale deployments. The widespread adoption of Anaconda in the data science community has made it a de facto standard for Python-based data science. Many tutorials, courses, and books assume that the user has an Anaconda installation, and many open-source projects provide conda packages to make installation easier.

NUMPY

NumPy, which stands for Numerical Python, is the fundamental package for scientific computing with Python. It provides support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays. At the core of the NumPy package is the ndarray object, which is a fast and memory-efficient multi-dimensional array. Unlike Python's built-in list data structure, which can contain elements of different data types, a NumPy array must contain elements of the same data type. This homogeneity allows for significant performance improvements, as the operations on the array can be vectorized and executed in compiled C or Fortran code. This is a key reason why NumPy is so much faster than pure Python for numerical computations.

One of the most powerful features of NumPy is its support for broadcasting. Broadcasting is the term used to describe how NumPy treats arrays with different shapes during arithmetic operations. Subject to certain constraints, the smaller array is "broadcast" across the larger array so that they have compatible shapes. This allows for very concise and efficient code, as it avoids the need for explicit loops. For example, you can add a scalar value to every element of an array with a single operation, or you can add a one-dimensional array to each row of a two-dimensional array. This ability to perform element-wise operations on arrays is a cornerstone of numerical computing in Python.

In addition to its powerful array object, NumPy also provides a wide range of mathematical functions. These include functions for linear algebra, Fourier analysis, and random number generation. Many of these functions are implemented in C or Fortran, which makes them very fast. The linear algebra module, for example, provides functions for matrix multiplication, finding the determinant of a matrix, and solving systems of linear equations. The random number generation module is also very powerful and can be used to generate random numbers from a variety of probability distributions. NumPy is the foundation of the scientific Python ecosystem. Many other popular data science libraries, such as Pandas, Matplotlib, and Scikit-learn, are built on top of NumPy and use its array object as their primary data structure. This means that if you want to do data science in Python, you need to have a good understanding of NumPy.

Pandas

Pandas is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation tool, built on top of the Python programming language. It is one of the most popular libraries for data science and is widely used for data cleaning, transformation, and analysis. The name "Pandas" is derived from the term "panel data," an econometrics term for multidimensional, structured data sets. The two primary data structures in Pandas are the Series and the DataFrame. A Series is a one-dimensional labeled array capable of holding any data type. It is similar to a NumPy array, but it has an associated index, which can be used to label the elements of the series. A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. You can think of it like a spreadsheet or a SQL table, or a dictionary of Series objects. It is the most commonly used pandas object.

One of the key features of Pandas is its ability to read and write data from a variety of formats, including CSV, Excel, SQL databases, and HDF5. This makes it easy to get your data into a DataFrame so that you can start working with it. Once the data is in a DataFrame, you can use a wide range of methods to clean, transform, and analyze it. For example, you can use Pandas to handle missing data, to filter and select subsets of your data, to group and aggregate your data, and to merge and join different datasets. The DataFrame object provides a rich set of methods for performing these operations, and the syntax is designed to be intuitive and expressive.

Another powerful feature of Pandas is its support for time-series data. Pandas has a comprehensive set of tools for working with dates, times, and time-indexed data. This makes it an ideal tool for analyzing financial data, sensor data, and any other type of data that has a time component. You can use Pandas to resample your time-series data to a different frequency, to calculate rolling statistics, and to perform time-zone conversions. The integration of Pandas with other data science libraries, such as NumPy, Matplotlib, and Scikit-learn, is also a major advantage. You can easily pass data between these libraries, which allows you to build powerful data analysis pipelines. For example, you can use Pandas to clean and prepare your data, and then use Scikit-learn to build a machine learning model on that data. The results of the model can then be visualized using Matplotlib.

MATPLOTLIP

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is one of the most widely used plotting libraries in the Python ecosystem and is the foundation for many other visualization libraries. Matplotlib provides a low-level, object-oriented API for creating a wide variety of plots, including line plots, scatter plots, bar charts, histograms, and more. The library is designed to be highly customizable, giving you complete control over every aspect of your plots, from the colors and line styles to the fonts and labels. This flexibility makes it possible to create publication-quality figures that can be tailored to your specific needs.

The core of the Matplotlib library is the pyplot module, which provides a simple and convenient interface for creating plots. The pyplot module is designed to be similar to MATLAB's plotting interface, which makes it easy for users who are familiar with MATLAB to get started with Matplotlib. The typical workflow for creating a plot with pyplot is to first create a figure and an axes object, and then to use the methods of the axes object to add elements to the plot, such as lines, points, and labels. Once you have created your plot, you can save it to a file in a variety of formats, including PNG, PDF, and SVG.

While the pyplot interface is great for creating simple plots quickly, the object-oriented API provides more control and flexibility. The object-oriented API allows you to interact with the different components of a plot, such as the figure, the axes, the lines, and the text, as individual objects. This makes it possible to create complex and highly customized plots. For example, you can use the object-oriented API to create plots with multiple subplots, to add annotations and arrows to your plots, and to create custom legends. The ability to create a wide variety of plots and the high degree of customization are the main reasons for Matplotlib's popularity. It is the go-to library for many data scientists and researchers who need to create high-quality visualizations for their work.

SEABORN

Seaborn is a Python data visualization library based on Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics. While Matplotlib provides a great deal of flexibility and control over the appearance of plots, it can be quite verbose and complex to create some of the more common statistical plots. Seaborn aims to simplify this process by providing a set of functions that are specifically designed for creating statistical visualizations. These functions are designed to work well with Pandas DataFrame objects, which makes it easy to create plots directly from your data.

One of the key features of Seaborn is its ability to create complex visualizations with just a single line of code. For example, you can create a scatter plot with a regression line, a box plot, or a violin plot with a single function call. Seaborn also provides a number of functions for visualizing the distribution of a dataset, such as histograms, kernel density estimates, and cumulative distribution functions. These functions are designed to be easy to use and to produce aesthetically pleasing plots with sensible defaults. The default color palettes and plot styles in Seaborn are also designed to be more visually appealing than the defaults in Matplotlib.

In addition to its high-level plotting functions, Seaborn also provides a number of tools for working with color palettes. The color_palette() function can be used to create a variety of different color palettes, and the set_palette() function can be used to set the default color palette for all of your plots. This makes it easy to create a consistent and visually appealing style for your visualizations. Seaborn is not a replacement for Matplotlib, but rather a complement to it. In fact, many of the functions in Seaborn are built on top of Matplotlib. This means that you can still use all of the power and flexibility of Matplotlib to customize your Seaborn plots. For example, you can use Matplotlib to change the labels, titles, and other properties of your Seaborn plots. The combination of Seaborn's high-level interface and Matplotlib's low-level customization makes it a powerful tool for creating a wide range of statistical visualizations.

TENSORFLOW

TensorFlow is a free and open-source software library for machine learning and artificial intelligence. It was developed by the Google Brain team and was first released in 2015. TensorFlow is one of the most popular deep learning frameworks and is used by researchers and developers around the world to build and train neural networks. The core of TensorFlow is a system for defining and running computations involving tensors. A tensor is a multi-dimensional array, similar to a NumPy array. The computations in TensorFlow are represented as a dataflow graph, where the nodes in the graph represent mathematical operations, and the edges represent the tensors that flow between them. This graph-based approach allows for a great deal of flexibility and makes it possible to run TensorFlow programs on a variety of different hardware platforms, including CPUs, GPUs, and TPUs.

One of the key features of TensorFlow is its support for automatic differentiation. This means that you can define a neural network as a series of mathematical operations, and TensorFlow will automatically compute the gradients of the network's parameters with respect to a loss function. This is a crucial step in the training process of a neural network, as it allows you to use gradient-based optimization algorithms, such as stochastic gradient descent, to update the network's parameters. The high-level API in TensorFlow, tf.keras, makes it easy to build and train neural networks. Keras is a user-friendly API that is designed for rapid prototyping. It provides a set of building blocks, such as layers, activation functions, and optimizers, that can be used to create a wide variety of neural network architectures.

TensorFlow also provides a number of tools for deploying machine learning models in production. TensorFlow Serving is a flexible, high-performance serving system for machine learning models, designed for production environments. It allows you to easily deploy your trained models and to serve predictions to your users. TensorFlow Lite is a lightweight version of TensorFlow that is designed for deploying models on mobile and embedded devices. It provides a set of tools that help you to optimize your models for size and performance. The large and active community around TensorFlow is another major advantage. There are a wealth of tutorials, examples, and pre-trained models available online, which makes it easy to get started with the framework. The community also provides support through forums and mailing lists, which can be a valuable resource when you are working on a new project.

PYTORCH

PyTorch is an open-source machine learning library based on the Torch library, used for applications such as computer vision and natural language processing. It was primarily developed by Facebook's AI Research lab (FAIR). It is known for its simplicity, flexibility, and Pythonic nature. PyTorch is often compared to TensorFlow, and while both are powerful deep learning frameworks, they have some key differences in their design and philosophy. One of the main differences is that PyTorch uses a dynamic computation graph, while TensorFlow traditionally used a static computation graph. In a static computation graph, you first define the entire computation graph, and then you run it. In a dynamic computation graph, the graph is defined on-the-fly as the computations are performed. This makes it easier to debug and to build more complex models, especially in natural language processing where the structure of the model can depend on the input data.

The core data structure in PyTorch is the Tensor, which is similar to a NumPy array. PyTorch tensors can be used on a GPU to accelerate computing. The library provides a rich set of operations for creating and manipulating tensors, and it also has a powerful automatic differentiation engine called autograd. The autograd module allows you to automatically compute the gradients of your tensors, which is essential for training neural networks. The torch.nn module in PyTorch provides a set of building blocks for creating neural networks, such as layers, loss functions, and optimizers. The API is designed to be intuitive and easy to use, and it allows you to define your models as Python classes.

PyTorch has a strong and growing community, and it has been adopted by many researchers and developers in both academia and industry. The documentation is excellent, and there are many tutorials and examples available online. The library also has a number of tools for production deployment, such as TorchScript, which allows you to serialize and optimize your models for deployment, and TorchServe, which is a flexible and easy-to-use tool for serving PyTorch models. The ease of use, flexibility, and strong community support have made PyTorch one of the most popular deep learning frameworks, and it is a great choice for both research and production.

DEVELOPMENT ENVIRONMENT SETUP

PREREQUISITES

Ensure you have the following components ready. Understanding their purpose will help in troubleshooting if issues arise.

- Administrator/Sudo Privileges: Required for installing software system-wide, such as the Python interpreter itself or NVIDIA drivers.
- A Modern Code Editor: An Integrated Development Environment (IDE) or code editor like VS Code or PyCharm is highly recommended. They provide features like code completion, debugging, and terminal integration, which significantly streamline development.
- NVIDIA GPU (For GPU Acceleration): Deep learning models involve massive matrix computations. A compatible NVIDIA GPU can perform these calculations in parallel, reducing training time from days to hours or even minutes. This is optional but highly recommended for any serious deep learning work.
- NVIDIA Drivers (For GPU Acceleration): These are the software that allows your operating system to communicate with your GPU. You must install the correct, up-to-date drivers from the NVIDIA website before proceeding with the CUDA toolkit installation.

NSTALL PYTHON

The foundation of our setup is the Python interpreter. We'll install it directly from the source to ensure we have the latest stable version.

Download Python:

- Navigate to the official Python downloads page.
- Download the installer for the latest stable release (e.g., Python 3.11.x). Avoid beta or release candidate versions for a stable development environment.

2. Install Python (Important Settings):

- Run the downloaded installer.
- On Windows: This is a crucial step. On the first screen of the installer, check the box that says "Add Python [version] to PATH." The PATH is an environment variable that your operating system uses to locate executables. Adding Python to the PATH allows you to run python and pip from any directory in your command prompt.

VERIFY INSTALLATION:

- Open a new terminal or command prompt window (to ensure it loads the updated PATH variable).
- Run the following commands to confirm that Python and its package manager, pip, are correctly installed and accessible.

Check Python version python
--version
Check Pip version (Python's package installer)
pip --version

Create a Virtual Environment

This is one of the most critical best practices in modern Python development. It prevents "dependency hell," where different projects on the same machine require conflicting versions of the same library.

Create a Project Directory:

- All your project files, including the isolated environment, will live here.

```
# Create a new directory and navigate into it
mkdir my_ml_project cd
my_ml_project
```

Create the Virtual Environment:

- This command uses Python's built-in venv module to create a new directory (we name it venv) that contains a copy of the Python interpreter, the standard library, and a private copy of pip.

```
python -m venv venv
```

Activate the Virtual Environment:

- Activating the environment modifies your current shell session's PATH to prepend the path to the venv directory. This means when you type python or pip, you are using the copies from within your venv folder, not the global ones.
- On Windows (Command Prompt/PowerShell):

```
.\venv\Scripts\activate source
venv/bin/activate
```

- Your command prompt will change to show (venv) at the beginning, confirming that the environment is active.

Deactivating the Environment: deactivate

Install Core ML/AI Libraries

With the virtual environment active, any package installed using pip will be placed inside the venv folder, keeping your global Python installation clean.

Common Data Science Libraries

These are the workhorses of any data science project.

- NumPy: The fundamental package for numerical computing, providing powerful N-dimensional array objects.

- Pandas: Provides high-performance, easy-to-use data structures (like the DataFrame) and data analysis tools.
- Matplotlib & Seaborn: Powerful libraries for creating static, animated, and interactive visualizations.

PyTorch (Deep Learning)

PyTorch is valued for its flexibility and Pythonic nature, making it a favorite in the research community.

1. Go to the PyTorch website.
2. Select Your Configuration: Choose the options that match your system. The "Compute Platform" is the most important choice:
   - CPU: For systems without a compatible NVIDIA GPU.
   - CUDA: Select this if you have an NVIDIA GPU. This will install a version of PyTorch that can offload computations to the GPU for massive speedups.

Run the Generated Command: The command includes PyTorch itself, along with torchvision (for popular computer vision datasets and models) and torchaudio (for audio tasks). # Example command, use the one from the website for your specific setup pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121

Verify PyTorch Installation:

- Start the Python interpreter by typing python in your terminal.
- This code confirms the installation and checks if PyTorch can communicate with your GPU.

```
import torch
print(f"PyTorch Version: {torch.__version__}")
cuda_is_available = torch.cuda.is_available()
print(f"CUDA Available: {cuda_is_available}") if
cuda_is_available:
  print(f"Device Name: {torch.cuda.get_device_name(0)}")
```

TensorFlow (Deep Learning)

TensorFlow, developed by Google, is known for its robust production deployment options and extensive ecosystem.

Install TensorFlow:

- Modern TensorFlow packages (tensorflow >= 2.x) bundle CPU and GPU support into one package. It will automatically detect and use a GPU if the necessary NVIDIA drivers and CUDA toolkit are correctly installed on your system.

pip install tensorflow

Verify TensorFlow Installation:

- Start the Python interpreter (python) and run the following code. It checks the version and explicitly lists physical GPU devices recognized by TensorFlow.

```
import tensorflow as tf
print(f"TensorFlow Version: {tf.__version__}") gpu_devices
= tf.config.list_physical_devices('GPU') print(f"GPU
Available: {len(gpu_devices) > 0}")
if gpu_devices:
  print(f"GPU Devices Found: {gpu_devices}") #
```

Type exit() to leave the Python interpreter **Install**

Web Frameworks

These frameworks are used to create user interfaces for your models, from simple APIs to full-fledged interactive web apps.

Flask

A "micro" web framework, meaning it provides the bare essentials for web development (like routing and request handling) and lets you choose any additional libraries you want. It's perfect for building REST APIs to serve model predictions.

Install: pip install Flask

Example (app.py):

```
from flask import Flask, jsonify
# Creates an instance of the Flask application app
= Flask(__name__)
# Defines a route. When a user visits "/predict", this function runs.
@app.route("/predict")
def predict():
  # In a real app, you would load a model and run inference here
```

```
    prediction_data = {'class': 'cat', 'confidence': 0.92}
    return jsonify(prediction_data) # Converts a Python dict to a JSON response
# This block ensures the server only runs when the script is executed directly
if __name__ == "__main__":
    app.run(debug=True) # Starts the dev server with debug mode on
```

Run: flask --app app run

(view at http://12.0.0.1:5000)

Streamlit

An open-source Python library that makes it easy to create and share beautiful, custom web apps for machine learning and data science. You write a simple Python script, and Streamlit renders it as an interactive web app.

Install: pip install streamlit

Example (st_app.py):

```
import streamlit as st
import pandas as pd

st.title('My First Streamlit App')
st.write("Streamlit turns Python variables into web content:") df
= pd.DataFrame({
  'first column': [1, 2, 3, 4],
  'second column': [10, 20, 30, 40]
})
# Streamlit magically knows how to render a Pandas DataFrame
st.write(df)
```

Run: streamlit run st_app.py**Django**

A high-level, "batteries-included" web framework for building large, scalable, and secure web applications. It comes with an Object-Relational Mapper (ORM), an admin panel, user authentication, and more, right out of the box.

Install: pip install Django

Start Project: This command creates the directory structure and files for a new Django

project.

django-admin startproject mydjangoproject .

Run Migrations: This command creates the initial database schema required by Django's built-in apps (like authentication).

python manage.py migrate

Run Server: Starts the lightweight development server. python manage.py runserver

(view at http://127.0.0.1:8000)

2. SYSTEM STUDY

2.1 EXISTING SYSTEM:

In the existing system, gender and age detection is mostly performed using manual observation or offline image analysis. Traditional methods rely on basic image processing techniques that detect facial features from static images. These systems are limited in accuracy, unable to process real-time video streams, and are often sensitive to lighting conditions, pose variations, and occlusions. As a result, they are not suitable for live applications such as surveillance, attendance monitoring, or interactive systems.

2.1.1 DISADVANTAGE:

- Cannot process real-time video efficiently.
- Accuracy is low due to lighting and pose variations.
- Manual observation or offline analysis is time-consuming.
- Limited scalability for large datasets or multiple camera feeds.
- Cannot handle occlusions like glasses, masks, or hats effectively.
-

2.2 PROPOSED SYSTEM

The proposed system performs real-time gender and age detection using facial images captured from live video streams. It uses machine learning and deep learning algorithms to analyze facial features and predict gender and approximate age accurately. The system is robust against varying lighting conditions, poses, and partial occlusions. It can be used in applications like smart surveillance, retail analytics, and interactive systems, providing fast, automated, and reliable detection.

2.2.1 ADVANTAGE:

- Real-time gender and age prediction from live video feed.
- High accuracy using advanced machine learning/deep learning models.
- Works under different lighting conditions and poses.
- Can handle partial occlusions (glasses, masks, hats).
- Automated and fast processing for practical applications.

3. SYSTEM DESIGN AND DEVELOPMENT

3.1 FILE DESIGN

The file design of the system organizes input and output data efficiently. Input files include images or video streams captured from cameras, while output files store predicted results such as detected faces, gender, and age information. Pre-trained model files and configuration files are maintained separately to ensure modularity. Temporary files for intermediate processing, logs, and error reports are also created to help monitor and debug the system effectively.

3.2 INPUT DESIGN

When large volumes of data are being handled, it's important that the items to be stored are selected easily and quickly. To accomplish this each data item must have a unique specification and must be related to other forms or items of data of the same type. The master file transaction is related to the same type id in the other table while using the primary key in p-id. A master file in product id is related to another table. There are various product id numbers in the various numbers in the various numbers of products.

The purpose of codes is to facilitate the identification and retrieval of items of information. The system analysts will find code structures will not always be the most suitable for efficient computer processing principles of code design. To accomplish this, each data item must have a unique identification and must be related to the other items of data of the data and same data type.

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps necessary to put transaction data into a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use while retaining privacy.

Input Design considered the following things:

- What data should be given as input?
- How should the data be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when errors occur.

3.3 CODE DESIGN

The code design is modular and structured to ensure clarity and reusability. The system consists of modules for image capture, face detection, feature extraction, and gender-age prediction. Each module is implemented as a separate class or function in Java/Python, allowing easy updates or replacements. The code follows object-oriented principles, and error handling is included to manage exceptions such as missing frames, low-quality images, or invalid inputs.

3.4 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other systems through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source of information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can be used easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.
2. Select methods for presenting information.
3. Create document, report, or other formats that contain information produced by the system. The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the Future.

- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

3.5 DATABASE DESIGN

The database design stores information related to users, captured images, and prediction results. Tables include user profiles, image metadata, and prediction logs, which help track historical predictions for analysis. Relationships are defined between users and their associated images, and indexing is used for faster retrieval of records. The database ensures data consistency, security, and easy integration with the prediction modules.

3.6 SYSTEM DEVELOPMENT

The system is developed in stages, starting with requirement analysis, followed by dataset preparation and model training. Face detection and feature extraction modules are integrated with real-time video capture for live prediction. The system is tested under various conditions, including different lighting, poses, and occlusions, to ensure robustness. Finally, the application is deployed with a user-friendly interface that displays live predictions, logs results, and allows easy access to historical data.

3.6.1 MODULES

3.6.2 MODULES DESCRIPTION

1. Image/Video Capture Module:

This module captures images or live video feed from cameras. It ensures that frames are read in real-time and passed to the face detection module. The module handles different camera resolutions and frame rates, and manages exceptions like disconnected cameras or unreadable frames.

2. Face Detection Module:

This module detects faces in each captured frame using techniques like Haar Cascades, HOG, or deep learning-based detectors (e.g., MTCNN). It isolates face regions for further analysis, ensuring detection under varying lighting, poses, and partial occlusions.

3. Feature Extraction Module:

This module extracts key facial features from the detected face regions, such as eyes, nose, mouth, and facial contours. These features are normalized and preprocessed to be suitable as input for the gender and age prediction models.

4. Gender Prediction Module:

This module uses a trained machine learning or deep learning model to predict the gender of each detected face. It outputs labels (male/female) and a confidence score, which can be used for real-time display or logging.

5. Age Prediction Module:

This module predicts the approximate age of the detected face using regression or classification models trained on age-labeled datasets. The predicted age is displayed along with the detected face in the video feed.

6. Result Display and Logging Module:

This module overlays the detected gender and age information on the live video feed for real-time visualization. It also logs predictions, images, timestamps, and confidence scores in the database for analysis and record-keeping.

7. Exception Handling Module:

This module handles errors such as missing frames, undetectable faces, or invalid inputs. It ensures the system runs smoothly without crashing and provides useful error messages for debugging.

4. SYSTEM TESTING AND IMPLEMENTATION

4.1 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

FUNCTIONAL TEST

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

WHITE BOX TESTING

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

BLACK BOX TESTING

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or

requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

UNIT TESTING:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

TEST STRATEGY AND APPROACH

Field testing will be performed manually and functional tests will be written in detail.

TEST OBJECTIVES

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

FEATURES TO BE TESTED

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

INTEGRATION TESTING

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

TEST RESULTS:

All the test cases mentioned above passed successfully. No defects encountered.

ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

TEST RESULTS:

All the test cases mentioned above passed successfully. No defects encountered.

4.2 IMPLEMENTATION:

Implementation is the stage of the project when the theoretical design is turned out into a working system. Thus it can be considered to be the most critical stage in achieving a successful new system and in giving the user, confidence that the new system will work and be effective.

The implementation stage involves careful planning, investigation of the existing system and it's constraints on implementation, designing of methods to achieve changeover and evaluation of changeover methods.

TEST RESULTS:

All the test cases mentioned above passed successfully. No defects encountered.

4.3 IMPLEMENTATION:

Implementation is the stage of the project when the theoretical design is turned out into a working system. Thus it can be considered to be the most critical stage in achieving a successful new system and in giving the user, confidence that the new system will work and be effective.

The implementation stage involves careful planning, investigation of the existing system and it's constraints on implementation, designing of methods to achieve changeover and evaluation of changeover methods.

5. FEASIBILITY STUDY

All projects are feasible given unlimited users and infinite time. Unfortunately the development is more likely to be plagued by a scarcity resources and difficult delivery date. It is both necessary and prudent to evaluate the feasibility of an object, no such limitation imposed that is not feasible as all the resources are easily available and time given was sufficient. Feasibility study was made according to four key considerations

TECHNICAL FEASIBILITY

Technical feasibility centres on the exiting computer and to what extent it can support the proposed addition. This involves financial considerations to accommodate technical enhancement. If the budget is a serious constraint, the project is judged as not feasible.

BEHAVIORAL FEASIBILITY

An estimate should be made of how strong a reaction the user staff is likely to have toward the development of computerized system. It is common knowledge the computer installation have something to do understandable that the introduction of a candidate system requires special effort to educate, sell and train the staff on new ways of considering business.

OPERATIONAL FEASIBILITY

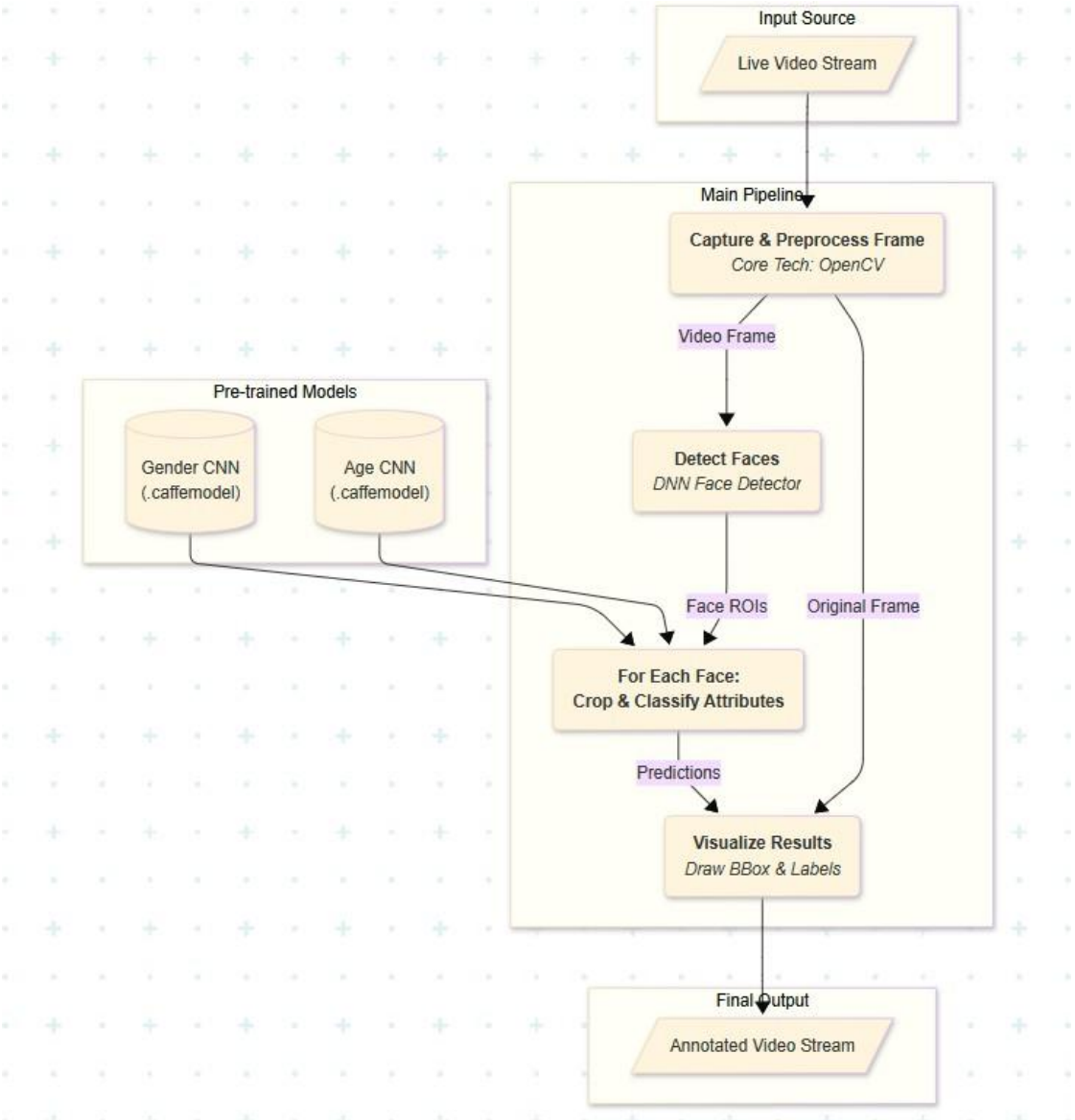
Operational feasibility is the method used to evaluate whether the proposed system is user friendly. They must be easy to handle, maintain and modify. The main objectives is that they should be enough flexible.

ECONOMICAL FEASIBILITY

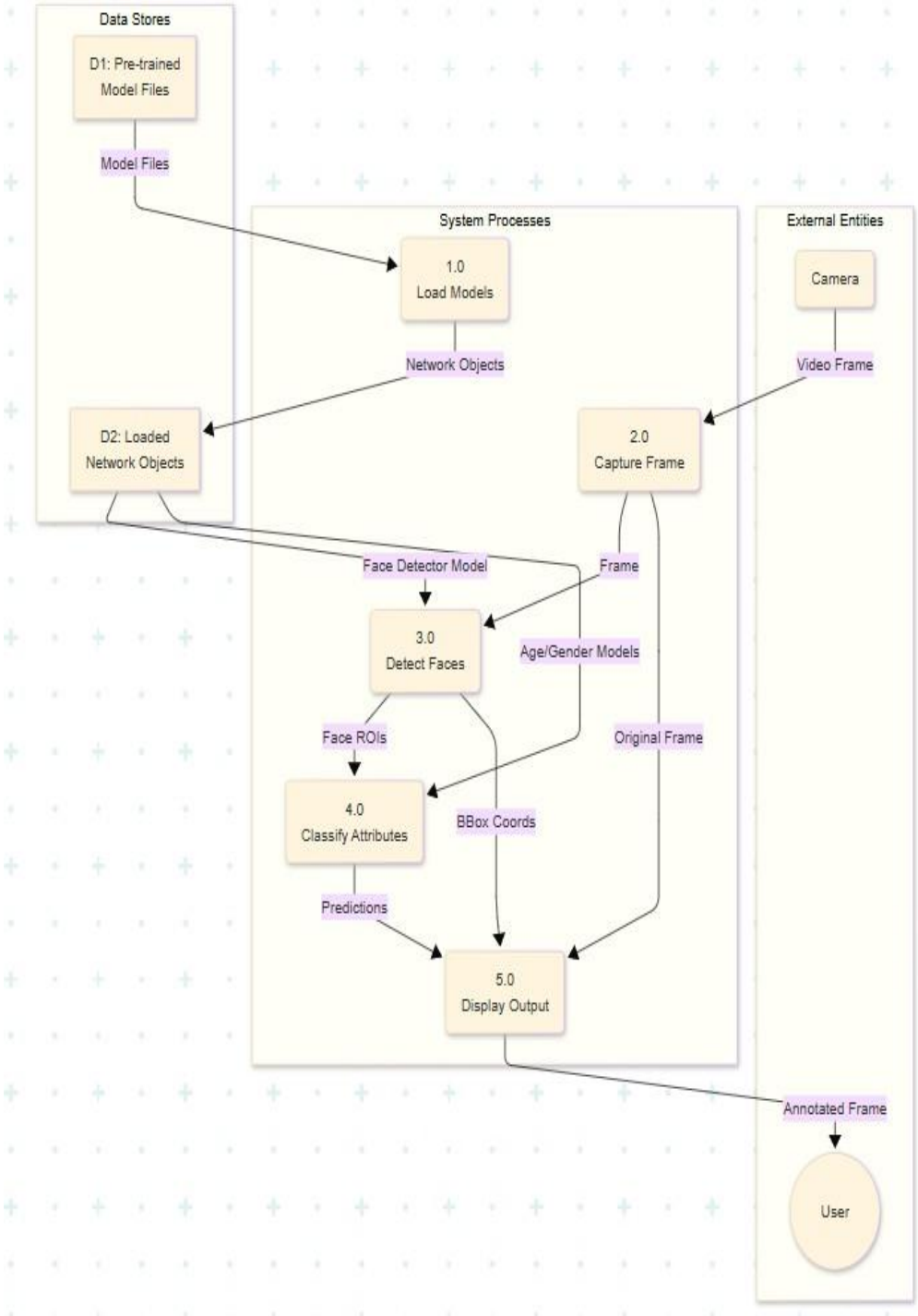
Economic analysis is the most frequently used method for evaluating the effectiveness of a system more commonly known as cost/benefit analysis, the procedures is made to determine the benefits and savings that are expected from system and compare them with costs. The result of comparison for found out and changed if needed. This is an ongoing effort that improves in accuracy at each phase of the system life cycle. Feasibility is both necessary and prudent to evaluate the feasibility of a project at the earliest possible time.

6. APPENDICES

A. SYSTEM ARCHITECTURE



DATA FLOW DIAGRAM



SAMPLE CODING

Main.py

```
from flask import Flask, render_template, Response import
cv2
import os
from model_loader import load_models from
face_detector import detect_faces from
classifier import classify_age_gender

app = Flask(__name__)

# --- Pre-load Models ---
# Load all the pre-trained models once when the application starts. try:
   face_net, age_net, gender_net = load_models()
   print("Models loaded successfully!")
except Exception as e:
   print(f"Error loading models: {e}")
   face_net, age_net, gender_net = None, None, None

# --- Constants from Model Training ---
MODEL_MEAN_VALUES = (78.4263377603, 87.7689143744, 114.895847746)
AGE_LIST = ['(0-2)', '(4-6)', '(8-12)', '(15-20)', '(25-32)', '(38-43)', '(48-53)', '(60-100)']
GENDER_LIST = ['Male', 'Female']

def process_frame(frame): """
   Processes a single frame from the video stream:
   1. Detects faces.
   2. For each face, classifies age and gender.
   3. Draws the results on the frame.
   Returns the annotated frame.
```

```
4. if face_net is None: return
   frame

5.
6. faces = detect_faces(face_net, frame)
7. for (startX, startY, endX, endY) in faces:
   face_roi = frame[startY:endY, startX:endX] if
   face_roi.size == 0:
8. continue
9.
10.
11.         age, gender = classify_age_gender(age_net, gender_net, face_roi,
   MODEL_MEAN_VALUES, AGE_LIST, GENDER_LIST)
12.         label = f"{gender}, {age}"
13.
14.
15.         cv2.rectangle(frame, (startX, startY), (endX, endY), (0, 255, 0), 2)
16.         cv2.putText(frame, label, (startX, startY - 10),
   cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2, cv2.LINE_AA)

17.
18.         return frame
19.
20.
21.         def generate_frames():
22.         """A generator function that yields frames from the
   webcam."""" camera = cv2.VideoCapture(0)
23.         if not camera.isOpened():
24.         print("Error: Could not open webcam.")
   return

25.
26.         while True:
27.         success, frame =
   camera.read() if not success:
28.         break else:
29.         annotated_frame = process_frame(frame)
```

```
30.         ret, buffer = cv2.imencode('.jpg',
annotated_frame) if not ret:
 31.           continue
32.
33.
34.         frame_bytes = buffer.tobytes()
yield (b'--frame\r\n'
35.         b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')
36.
37.
 38.          camera.rele
                    ase()
39.
40.
41.         # --- Flask Routes ---
42.         @app.rou
te('/') def index():
43.         """Renders the main
page."""  return
render_template('index.html')

44.
45.         @app.route('/video
_feed') def video_feed():
46.         """Provides the video stream from the webcam."""
47.         return Response(generate_frames(), mimetype='multipart/x-mixed-
replace; boundary=frame')

48.
49.         if __name____== '_
main__':
app.run(debug=True)

50.
51.         Model_lo
ader.py import
cv2 import os
```

```
52.
53.         def
    load_models(): """
54.         Loads the face detection, age, and gender classification
    models. Returns the network objects for each model.
55.         """
56.         # Define paths to the model files.
57.         # These files should be in a 'models' directory in your project.
    face_proto = os.path.join('models', 'opencv_face_detector.pbtxt')
    face_model = os.path.join('models', 'opencv_face_detector_uint8.pb')

    age_proto = os.path.join('models', 'age_deploy.prototxt')
    age_model = os.path.join('models', 'age_net.caffemodel')

    gender_proto = os.path.join('models', 'gender_deploy.prototxt')
    gender_model = os.path.join('models', 'gender_net.caffemodel')

    # Check if model files exist
    required_files = [face_proto, face_model, age_proto, age_model, gender_proto,
gender_model]
    for file_path in required_files:
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"Model file not found: {file_path}. Please download the
models and place them in the 'models' directory.")

    # Load the networks using OpenCV's DNN module
    print("Loading face detection model...")
    face_net = cv2.dnn.readNet(face_model, face_proto)

    print("Loading age classification model...")  age_net
    = cv2.dnn.readNet(age_model, age_proto)
```

```
    print("Loading gender classification model...")
    gender_net = cv2.dnn.readNet(gender_model, gender_proto)

    return face_net, age_net, gender_net


Face_detector.py import
cv2
import numpy as np
def detect_faces(net, frame, confidence_threshold=0.7):
    """
    Detects faces in a given frame.

    Args:
        net: The loaded face detection model. frame:
        The input image/frame.
        confidence_threshold: The minimum probability to filter weak detections.

    Returns:
        A list of bounding boxes for detected faces.
    """
    # Get the frame's dimensions (h, w)
    = frame.shape[:2]

    # Create a blob from the image and pass it through the network
    blob = cv2.dnn.blobFromImage(frame, 1.0, (300, 300), (104.0, 177.0, 123.0))
    net.setInput(blob)
    detections = net.forward()

    faces = []
```

```
    # Loop over the detections
    for i in range(0, detections.shape[2]):
        # Extract the confidence (i.e., probability) associated with the prediction
        confidence = detections[0, 0, i, 2]

        # Filter out weak detections
        if confidence > confidence_threshold:
            # Compute the (x, y)-coordinates of the bounding box for the object
            box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
            (startX, startY, endX, endY) = box.astype("int")

            # Ensure the bounding box is within the frame dimensions
            startX = max(0, startX) startY
            = max(0, startY) endX =
            min(w - 1, endX) endY =
            min(h - 1, endY)

            # Add the bounding box to our list of faces if
            startX < endX and startY < endY:
                faces.append((startX, startY, endX, endY))

    return faces
```

Classifier.py import
cv2

```
def classify_age_gender(age_net, gender_net, face_roi, model_mean_values, age_list,
gender_list):
    """
    Classifies age and gender for a given face Region of Interest (ROI).
```

```
    Args:
        age_net: The loaded age classification model.
        gender_net: The loaded gender classification model.
        face_roi: The cropped image of the face.
        model_mean_values: Mean values for image normalization.
        age_list: List of age bracket labels.
        gender_list: List of gender labels.

    Returns:
        A tuple containing the predicted age and gender strings. """
    # Create a blob from the face ROI. This preprocesses the image
    # to the format the Caffe models expect (227x227 pixels, mean subtraction).
    blob = cv2.dnn.blobFromImage(face_roi, 1.0, (227, 227), model_mean_values,
swapRB=False)

    # --- Predict Gender ---
    gender_net.setInput(blob) gender_preds =
    gender_net.forward()
    # Get the gender with the highest confidence
    gender = gender_list[gender_preds[0].argmax()]

    # --- Predict Age ---
    age_net.setInput(blob) age_preds =
    age_net.forward()
    # Get the age bracket with the highest confidence age
    = age_list[age_preds[0].argmax()]

    return age, gender
```

```
Analyze_image.py import
cv2
import argparse
import os
from model_loader import load_models from
face_detector import detect_faces from
classifier import classify_age_gender

MODEL_MEAN_VALUES = (78.4263377603, 87.7689143744, 114.895847746)
AGE_LIST = ['(0-2)', '(4-6)', '(8-12)', '(15-20)', '(25-32)', '(38-43)', '(48-53)', '(60-100)']
GENDER_LIST = ['Male', 'Female']

def main(args):
  # --- 1. Load Models ---try:
    face_net, age_net, gender_net = load_models()
    print("Models loaded successfully!")
  except FileNotFoundError as e:
    print(f"Error: {e}")
    print("Please ensure the model files are in the 'models' directory.")
    return

  # --- 2. Read Input Image ---
  if not os.path.exists(args.image):
    print(f"Error: Input image not found at '{args.image}'")
    return

  frame = cv2.imread(args.image) if
  frame is None:
    print(f"Error: Could not read the image file '{args.image}'")
    return
```

```
    # --- 3. Detect Faces ---
    faces = detect_faces(face_net, frame) if
    not faces:
        print("No faces detected in the image.") return

    print(f"Detected {len(faces)} faces.")

    # --- 4. Classify and Annotate ---
    for (startX, startY, endX, endY) in faces: face_roi
        = frame[startY:endY, startX:endX] if
        face_roi.size == 0:
            continue

        age, gender = classify_age_gender(age_net, gender_net, face_roi,
MODEL_MEAN_VALUES, AGE_LIST, GENDER_LIST)
        label = f"{gender}, {age}"

        # Draw bounding box and label on the frame
        cv2.rectangle(frame, (startX, startY), (endX, endY), (0, 255, 0), 5)
        cv2.putText(frame, label, (startX, startY - 10), cv2.FONT_HERSHEY_SIMPLEX, 5, (0,
255, 0), 6, cv2.LINE_AA)

    # --- 5. Save and/or Display Output ---#
    Construct the output path
    output_filename = "output_" + os.path.basename(args.image)
    output_path = os.path.join(os.path.dirname(args.image), output_filename)

    # Save the annotated image
    cv2.imwrite(output_path, frame) print(f"Result
    saved to: {output_path}")
```

```
    # Display the result in a window  cv2.imshow("Age
    and Gender Detection", frame) print("Press any key
    to close the image window.") cv2.waitKey(0)
    cv2.destroyAllWindows()


if __name__== '__main__':
    # --- Command-Line Argument Parsing ---
    parser = argparse.ArgumentParser(description='Age and Gender Detection from an
image.')
    parser.add_argument('-i', '--image', type=str, required=True, help='Path to the input image
file.')

    args = parser.parse_args() main(args)
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Real-Time Age & Gender Detection</title>
    <!-- Tailwind CSS for styling -->
    <script src="https://cdn.tailwindcss.com"></script>
    <style>
        /* Custom styles for a cleaner look */ body {
            font-family: 'Inter', sans-serif; display:
            flex;
            align-items: center; justify-
            content: center; min-height:
            100vh;
```

```
    }
    .container {
      max-width: 900px;
    }
    .video-container {
      background-color: #1a202c; /* Dark background for video */
      border: 1px solid #4a5568;
      width: 50%;
      margin: 0 auto; display:
      flex;
      justify-content: center;
    }
  </style>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;700&display=swap"
rel="stylesheet">
</head>
<body class="bg-gray-900 text-white">

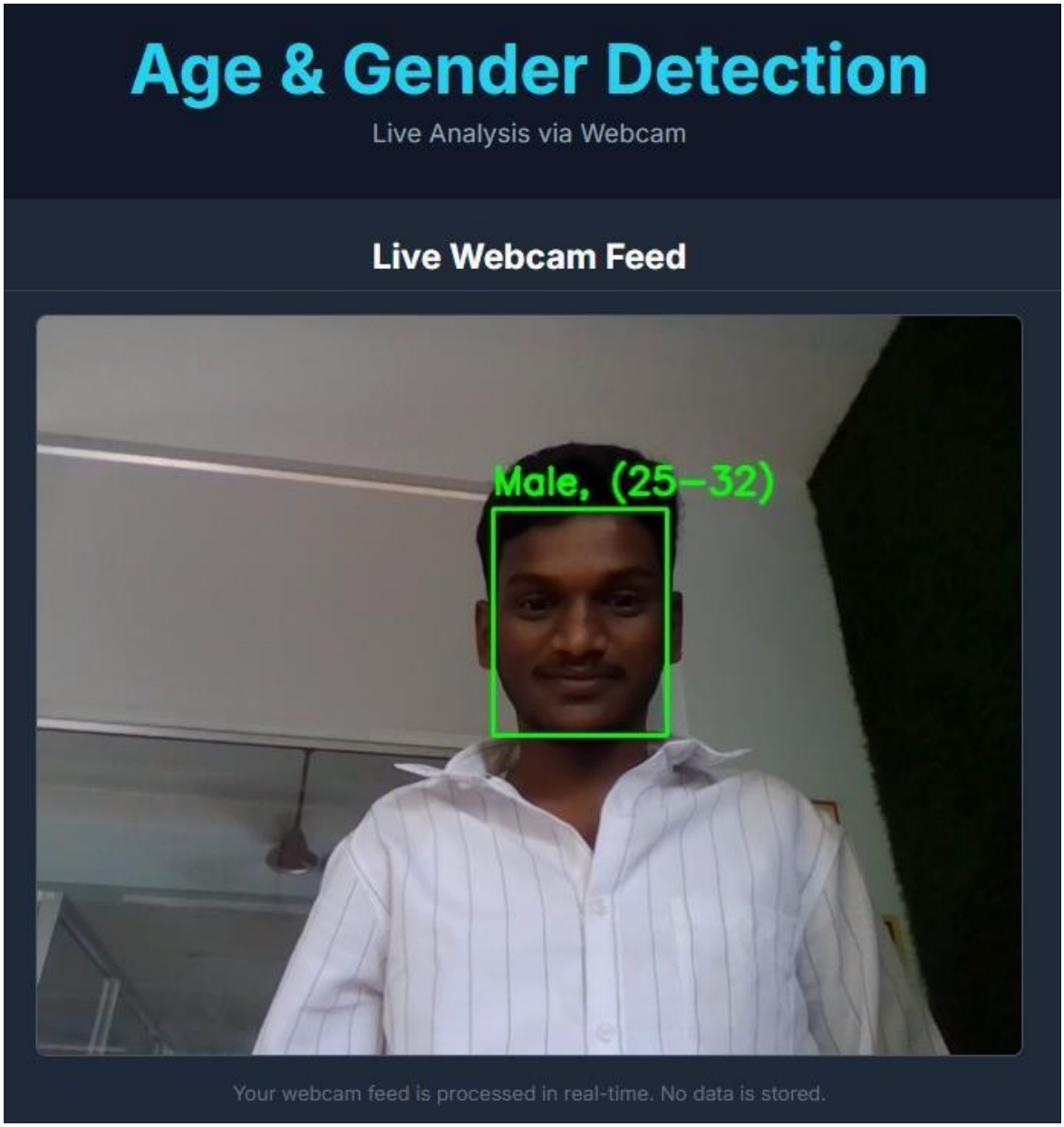
  <div class="container mx-auto p-4 md:p-8"><header class="text-center mb-8">
    <h1 class="text-4xl md:text-5xl font-bold text-cyan-400">Age & Gender
Detection</h1>
    <p class="text-lg text-gray-400 mt-2">Live Analysis via Webcam</p>
  </header>
  <div class="bg-gray-800 p-6 m-5 rounded-xl shadow-2xl">
    <h2 class="text-2xl font-semibold mb-4 text-center border-b border-gray-700 pb-
2">Live Webcam Feed</h2>
    <div class="video-container rounded-lg overflow-hidden mx-auto">
      <img src="{{ url_for('video_feed') }}" style="width:100%;" alt="Webcam Feed">
```

```
      </div>
      <p class="text-center text-gray-500 mt-4 text-sm">Your webcam feed is processed in real-time. No data is stored.</p>
   </div>

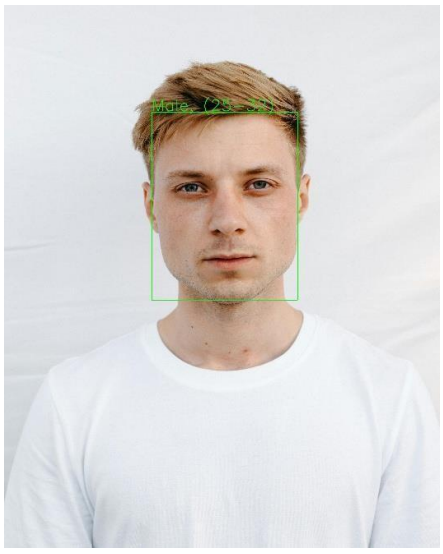
   <footer class="text-center mt-12 text-gray-500 text-sm">
      <p>Project based on the paper "Real-time Gender and Age Detection from Facial Images".</p>
   </footer>

</div>
</body>
</html> Requirements.txt
blinker==1.9.0 click==8.3.0
colorama==0.4.6
Flask==3.1.2
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.2
numpy==2.2.6
opencv-python==4.12.0.88
Werkzeug==3.1.3
```

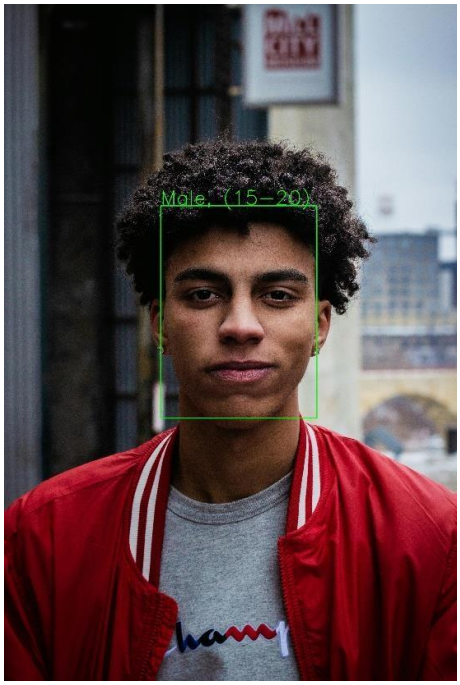
C. SAMPLE INPUT AND OUTPUT






Male, (25–32)




Male, (15–20)




Male, (4-6)

7.CONCLUSION

This project successfully designed a complete and highly efficient system for real-time gender and age detection from a live video feed. By adopting a hybrid, multi-stage pipeline that leverages pre-trained, optimized deep learning models within the OpenCV DNN module, the system effectively balances the competing demands of accuracy and computational efficiency. The core achievement of this project is the demonstration of a practical, end-to-end application that can perform complex demographic analysis on standard CPU hardware without the need for extensive model training or expensive GPUs. The use of a dedicated deep learning model for face detection and separate, specialized CNNs for age and gender classification proved to be a robust and effective strategy.

The project provides a comprehensive blueprint for building real-world computer vision applications through the intelligent integration of existing, state-of-the-art components. Future enhancements could involve replacing the Caffe models with newer, even more efficient models in the ONNX format, or expanding the system to classify other facial attributes like emotions or accessories. The lightweight nature of the pipeline also makes it a prime candidate for deployment on embedded systems or edge devices. In conclusion, this project serves as a powerful case study in practical AI engineering, delivering a functional and interactive system that solves a challenging computer vision problem in a highly accessible and performant manner.

REFERENCE

Book Reference

- **Erik Fransén, Deep Learning for Computer Vision, Packt Publishing, 2020.**
- **Adrian Rosebrock, Deep Learning for Computer Vision with Python, PyImageSearch, 2017.**
- **Simon Haykin, Neural Networks and Learning Machines, Pearson, 2009.**
- **Rafael C. Gonzalez, Richard E. Woods, Digital Image Processing, Pearson, 2018.**

Website References

- **OpenCV Documentation – https://opencv.org/**
- **PyImageSearch – https://www.pyimagesearch.com/**
- **Kaggle Datasets (Face Images) – https://www.kaggle.com/datasets**
- **Towards Data Science – https://towardsdatascience.com/**